On the Potential of Normalized TSP Features for Automated Algorithm Selection

Jonathan Heins
Statistics and Optimization
University of Münster
Münster, Germany
jonathan.heins@uni-muenster.de

Jakob Bossek
Statistics and Optimization
University of Münster
Münster, Germany
jakob.bossek@wi.uni-muenster.de

Janina Pohl
Statistics and Optimization
University of Münster
Münster, Germany
janina.pohl@uni-muenster.de

Moritz Seiler
Statistics and Optimization
University of Münster
Münster, Germany
moritz.seiler@wi.uni-muenster.de

Heike Trautmann
Statistics and Optimization
University of Münster
Münster, Germany
trautmann@wi.uni-muenster.de

Pascal Kerschke
Big Data Analytics in Transportation
TU Dresden
Dresden, Germany
pascal.kerschke@tu-dresden.de

ABSTRACT

Classic automated algorithm selection (AS) for (combinatorial) optimization problems heavily relies on so-called instance features, i.e., numerical characteristics of the problem at hand ideally extracted with computationally low-demanding routines. For the traveling salesperson problem (TSP) a plethora of features have been suggested. Most of these features are, if at all, only normalized imprecisely raising the issue of feature values being strongly affected by the instance size. Such artifacts may have detrimental effects on algorithm selection models. We propose a normalization for two feature groups which stood out in multiple AS studies on the TSP: (a) features based on a minimum spanning tree (MST) and (b) a k-nearest neighbor graph (NNG) transformation of the input instance. To this end we theoretically derive minimum and maximum values for properties of MSTs and k-NNGs of Euclidean graphs. We analyze the differences in feature space between normalized versions of these features and their unnormalized counterparts. Our empirical investigations on various TSP benchmark sets point out that the feature scaling succeeds in eliminating the effect of the instance size. Eventually, a proof-of-concept AS-study shows promising results: models trained with normalized features tend to outperform those trained with the respective vanilla features.

CCS CONCEPTS

• **Mathematics of computing** → *Graph theory*; • **Computing methodologies** → *Supervised learning*.

KEYWORDS

Automated algorithm selection, traveling salesperson problem (TSP), normalization, instance features, graph theory

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FOGA '21, September 6–8, 2021, Virtual Event, Austria
© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8352-3/21/09…$15.00
https://doi.org/10.1145/3450218.3477308

ACM Reference Format:
Jonathan Heins, Jakob Bossek, Janina Pohl, Moritz Seiler, Heike Trautmann, and Pascal Kerschke. 2021. On the Potential of Normalized TSP Features for Automated Algorithm Selection. In *Foundations of Genetic Algorithms XVI (FOGA '21), September 6–8, 2021, Virtual Event, Austria.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3450218.3477308

1 INTRODUCTION

In the context of per-instance automated algorithm selection (AS), initiated by Rice in the 1970s [25], the core idea is to predict – by means of machine learning models termed *selectors* in this field – the most likely best-performing solver for a given problem instance $I \in \mathcal{I}$ from a set $\mathcal{A}$ of at least two candidate solvers (see [13, 15] for recent surveys on AS). In classic AS approaches the instance space $\mathcal{I}$ in the selector-mapping $S : \mathcal{I} \to \mathcal{A}$ is replaced by a p-dimensional feature-space $\mathcal{F} \subset \mathbb{R}^p$, and each instance is mapped to the feature space by a computationally low-demanding function $f : \mathcal{I} \to \mathcal{F}$. The selector can be replaced by $S : \mathcal{F} \to \mathcal{A}$.

The Euclidean traveling salesperson problem (TSP) is a prominent NP-hard combinatorial optimization problem of huge practical relevance, e.g., for the fabrication of printed circuit boards as well as for transportation and logistics applications. The problem can be described as follows: we are given a complete undirected graph $G = (V, E)$ with $n = |V|$ nodes, node set $V = \{1, \ldots, n\}$ and pairwise distances $d(i, j), 1 \leq i, j \leq n$ between the nodes which are determined by the Euclidean metric. The goal is to compute a tour of minimal length that visits each city exactly once and eventually returns to the start of the tour. Formally, we seek a permutation $\pi : V \to V$ that minimizes the cost function

$$c(\pi) = d(\pi(n), \pi(1)) + \sum_{i=1}^{n-1} d(\pi(i), \pi(i+1)).$$

The TSP has been an intriguing problem in research for decades. It has ever since served as a test-bed for algorithmic ideas ultimately culminating in the development of the highly sophisticated exact TSP solver concorde [1] and several very well-performing heuristics such as the local-search algorithm LKH [9] and the genetic algorithm EAX [19, 20]. Likewise, in the past decade, during the rise of machine learning and more precisely AS-related research, the TSP continued to attract researchers [4–7, 11, 14, 16, 18, 21, 22, 28, 29].

In order to numerically characterize TSP problems, hundreds of features were developed by several groups [11, 18, 22]. Exemplary features that stand out in several studies as crucial performance-discriminating features, are based on properties of a minimum spanning tree (MST) of the graph at hand (e.g., summary statistics of the depth of nodes in the MST) or statistics of weakly/strongly connected components (CCs) of a so-called k-nearest-neighbor graph transformation (k-NNGs) of the source graph, e.g., the number of CCs or the size of the largest CC. However, until now, those features were used in an unnormalized way which may be detrimental and is certainly against intuition. As an example say we have two graphs G_1 with 30 nodes and G_2 with 3 000 nodes respectively. The number of weakly connected components in the 2-NNG may be up to 10 for G_1 and 1 000 for G_2 as we shall see later, although the pattern of weakly connected components in the 2-NNG are the same for both graphs, i.e., a collection of 3-cliques.[1] Such artifacts may induce a negative impact on algorithm selection due to features being dependent on the instance size. The goal of this paper is to introduce normalized versions of established TSP features

$$f_{\text{norm}} : \mathcal{I} \to \mathcal{F}_{\text{norm}} \subseteq [0,1]^p.$$

The respective normalization is based on the theoretical derivation of minimum and maximum values of MST- and k-NNG features which proved to have considerable discriminating impact on AS models in previous works [14, 22]. This is a substantial and mathematically sound extension of preliminary normalization efforts [22], in which the feature values have been divided by the instance size.

We will show that our normalized features not only improve understanding of feature and algorithm performance relations but also allow for constructing high-performing AS models on the Euclidean TSP across instance sizes. Section 2 provides the theoretical concept of our normalization approach together with the respective normalized feature variants, followed by an overview of the considered instance sets and inexact TSP solvers in Section 3. An exploratory illustration of the feature normalization effects on widely used and commonly accepted TSP instance sets [14] is presented in Section 4. A proof-of-concept AS study in Section 5 illustrates the promising potential of the introduced feature sets as input to automated algorithm selection models. Eventually, Section 6 summarizes the paper and discusses future work.

2 NORMALIZING FEATURES

In this section we derive theoretical results to normalize specific features. In abstract language, given a TSP feature value $f \in \mathbb{R}$ for a Euclidean graph G, we seek for lower and upper bounds $f_{\min}$ and $f_{\max}$ for the respective feature among all Euclidean graphs in order to calculate the *normalized* feature

$$f_{\text{norm}} := \frac{f - f_{\min}}{f_{\max} - f_{\min}} \in [0,1].$$

For this study, we do not consider all 200+ TSP features from the literature for two simple reasons: (1) space limitations, and (2) we do not have formal proofs for all feature normalizations (yet). Nevertheless, we do emphasize that a normalization of all features is desirable in future. Here, we focus on two feature groups that have proven to be very promising in discriminating solver performance in past TSP algorithm selection studies [14, 22], namely features based on a minimum spanning tree of the input graph (MST features) and a k-nearest neighbor graph transformation (NNG features). Implementations of the (normalized) features are available within the package `salesperson` [3] for the statistical programming language R [23].[2]

2.1 Preliminaries

A graph $G = (V, E)$ consists of a set of nodes $V = \{v_1, \ldots, v_n\}$ and a set of edges E. For brevity we denote by $n := |V|$ and $m := |E|$ the respective sizes. In a directed graph each edge (u, v) between nodes $u, v \in V$ has an orientation. In contrast, in undirected graphs there is no orientation which is highlighted by the set-style notation $\{u, v\}$. Each node $v \in V$ is associated with Euclidean coordinates (x_v, y_v) in the Euclidean plane. A function $d : V \times V \to \mathbb{R}^+$ describes distances/travel costs between pairs of nodes. The distance $d(u, v)$ between two nodes $u, v \in V$ corresponds to the Euclidean distance of the node coordinates of u and v in the plane. A *directed path* is a sequence of nodes $v_1, \ldots, v_l$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i < l$. In an undirected path, the corresponding undirected edges need to exist. For $U \subset V$ we define by $G[U] := (U, \{e \in E \,|\, |e \cap U| = 2\})$ the *induced subgraph* of G given U; $G[U]$ contains all nodes from U and all edges which link each two nodes in U. A graph is *(strongly) connected* if every node is reachable via a (directed) path from every other node. A subset of nodes $C \subset V$ is called a *strongly connected component* if the induced subgraph $G[C]$ is strongly connected and there is no superset of C that is strongly connected. Likewise, $C \subset V$ forms a *weakly connected component* if the induced subgraph is maximally weakly connected where the latter means that each node is reachable from any other node if edge orientations are neglected. A *spanning tree* of G is a acyclic subgraph of G covering all nodes. A *minimum spanning tree* (MST) is a spanning tree with minimum sum of edge weights across all possible spanning trees.

2.2 k-Nearest-Neighbor Graph Based Features

An important group of TSP features relies on characteristics of local nearest-neighbor relations, i.e., nodes and their k nearest neighbor nodes. This feature set was introduced by Pihera & Musliu [22] and is based on a k-NNG transformation of the input graph. Below, we give a formal definition following the notation used in [22].

Definition 2.1 (k-nearest-neighbor graph). Let $G = (V, E)$ be a graph with distance function $d : V \times V \to \mathbb{R}^+$. For $1 \leq i \leq n-1$ let $N(v, i)$ be the i-th nearest neighbor of node $v \in V$. Further denote by $E_k^d = \{(v, N(v, i)) \,|\, v \in V, 1 \leq i \leq k\}$ and $E_k^u = \{\{v, N(v, i)\} \,|\, v \in V, 1 \leq i \leq k\}$ the set of directed/undirected edges between nodes and their $1 \leq k \leq n-1$ nearest neighbor nodes. Then we call $G_k^d = (V, E_k^d)$ and $G_k^u = (V, E_k^u)$ the directed/undirected *k-nearest-neighbor graph* (k-NNG).

Figure 1 gives an impression of k-NNGs for different values of k on instance `a280` from the TSPLIB [24]. Given the directed and undirected k-NNG for a fixed k, this feature group consists of the number of weakly/strongly connected components and summary statistics of the number of nodes across weakly/strongly connected

[1] A k-clique in a graph $G = (V, E)$ is a subset of nodes $C \subseteq V$, $|C| = k$ such that each two nodes in C are linked by an edge.

[2] GitHub repository: https://github.com/jakobbossek/salesperson/.

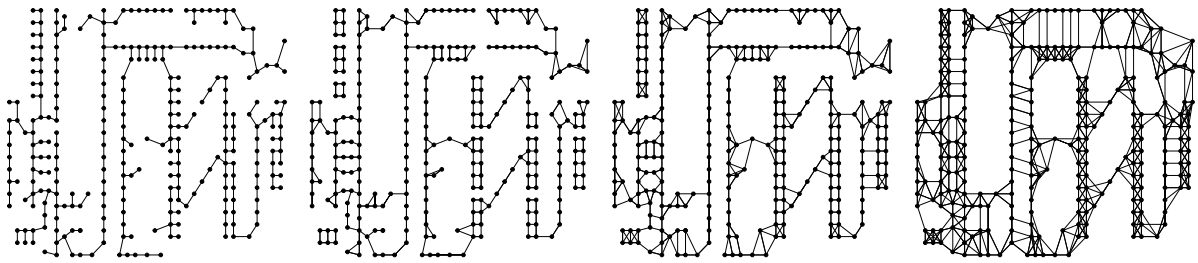


Figure 1: From left to right: a MST, 1-NNG, 3-NNG and 5-NNG of TSPLIB instance `a280`.

components, i.e., the mean, median, minimum, maximum and the span. It is obvious that these take values that highly depend on the graph size n. It should be noted that Pihera & Musliu also introduce normalized versions of these features, where normalization is achieved by dividing with instance size n. Since n is not the maximum value achievable for most of the features (see our derivations below) we do not consider this step as a sufficiently precise normalization. Therefore, we next derive results on the minimum and maximum possible values in order to normalize all NNG-features mathematically sound.

Theorem 2.2. *For a complete undirected graph $G = (V, E)$ and $k \geq 2$ the minimal number of weakly/strongly connected components in G_k^d and G_k^u equals one.*

Proof. Consider a graph where nodes $v_1, \ldots, v_n$ are placed circular with equidistant distances between neighbor nodes. Then for $k = 2$, in G_k^d, each node is linked to its two neighbors on the cycle via directed edges. In consequence, in G_k^d each node can be reached from any other node, e.g., by following edges in clock-wise direction. Thus, the number of strongly connected components equals one and so does the number of weakly connected components. Adding further edges in case $k > 2$ does not change the connectivity and therefore the results transfer to the general case. □

Theorem 2.3. *For a complete undirected graph $G = (V, E)$ and $k \geq 2$ the maximum number of weakly connected components in G_k^u is $\lfloor n/(k+1) \rfloor$.*

Proof. Each weakly connected component contains at least $k+1$ nodes since every node is linked to its k nearest neighbors and reachability is symmetric in undirected graphs. Thus, the number of weakly components is at most $\lfloor n/(k+1) \rfloor$. □

An example for a Euclidean graph with a maximal number of weakly connected components is a graph with $\lfloor \frac{n}{k+1} \rfloor$ non-overlapping clusters where the inter-cluster distance is higher than the distance between any pair of nodes within a cluster.

The following lemma will prove useful to derive the maximum number of strongly connected components in G_k^d.

Lemma 2.4. *For any $0 < q < 1/2$, $L \geq 0$ and $k \geq 1$ it holds that $q^L > q^{L+1} + q^{L+2} + \ldots + q^{L+k}$.*

Proof. Let $L \geq 0$ and $k \geq 1$ and $0 < q < 1$. Then we have

$$\begin{aligned}\sum_{j=1}^{k} q^{L+j} &= q^L \left(\sum_{j=1}^{k} q^j \right) = q^L \left(\sum_{j=0}^{k} q^j - 1 \right) \\ &= q^L \left(\frac{1-q^{k+1}}{1-q} - \frac{1-q}{1-q} \right) = q^L \left(\frac{q-q^{k+1}}{1-q} \right) \\ &= \frac{q^{L+1}\overbrace{(1-q^k)}^{<1}}{1-q} \leq \frac{q^{L+1}}{1-q}\end{aligned}$$

where we used the geometric sum $\sum_{j=0}^{n} q^j = \frac{1-q^{n+1}}{1-q}$ for $q \neq 1$ in the third equation. Eventually $q^{L+1}/(1-q) < q^L \Leftrightarrow q < 1-q \Leftrightarrow q < 1/2$. □

Theorem 2.5. *For a complete undirected graph $G = (V, E)$ and $k \geq 1$ the maximum number of strongly connected components in G_k^d equals $n - k$.*

Proof. We first construct a complete Euclidean graph $G = (V, E)$ with exactly $n - k$ strongly connected components in G_k^d and later argue, that more than $n - k$ components are impossible. Consider an n-vertex path where n nodes $v_1, \ldots, v_n$ are aligned from left to right. Let $C = 1/3$ be a constant. Place the nodes co-linear such that $d(v_i, v_{i+1}) = C^i$ for $1 \leq i < n$. In consequence, $d(v_1, v_2) > d(v_2, v_3) > \ldots > d(v_{n-1}, v_n)$. We show that for $1 \leq i \leq n-k$ the k nearest neighbors are $v_{i+1}, \ldots, v_{i+k}$. Since the distances $d(v_i, v_{i+1})$ are monotonically decreasing this holds if and only if $d(v_i, v_{i+k}) \leq d(v_{i-1}, v_i)$. We have

$$d(v_i, v_{i+k}) = \sum_{j=0}^{k-1} d(v_{i+j}, v_{i+j+1}) = \sum_{j=0}^{k-1} C^{i+j} < C^{i-1} = d(v_{i-1}, v_i)$$

where the first equality is due to the graph being Euclidean and the inequality follows from Lemma 2.4.

Hence, building the directed k-NNG up to node v_{n-k-1} does not introduce any cycles. However, for the latter nodes $v_{n-k+1}, \ldots, v_n$ there need to be right-to-left (backward) edges to fill up the set of k nearest neighbors which introduces directed cycles into the k-NNG up to node v_{n-k-1}. Hence, there is one component of size $n - (n-k-1) = k+1$ which results in a total number of $n-k$ strongly connected components. See Figure 2 for a visualization with $k = 2$.

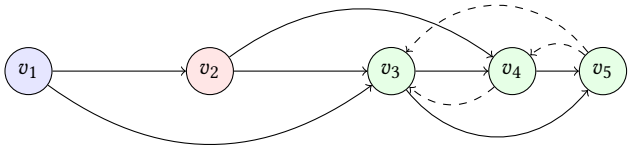


Figure 2: Illustration of the 2-NNG graph of a 5-vertex path with strictly decreasing consecutive distances. Solid lines are forward edges, dashed lines are backward edges. Colors indicate membership to strongly connected components; in total there are $5-2=3$ here.

Eventually, we argue that there cannot exist a graph G where G_k^d has strictly more than $n-k$ strongly connected components. To this end it is sufficient to show that there has to be one component with at least $k+1$ nodes. Consider a k-NNG $G_k^d = (V_G, E_G)$ with strongly connected components containing at most $l < k+1$ nodes. For the nodes v_i in a component C of G_k^d with l or less nodes it holds that there are at least $k+1-l$ edges $(v_i, v_j) \in E_G$ with $v_j \notin C$. There cannot be any edge $(j,i) \in E_G$ connecting any v_j with any v_i since this would introduce a cycle and therefore mean that $|C| > l$. Thus, v_j is connected to k other nodes and only $l-1$ of those nodes can be connected back to v_j. Continuing this with no connections back to any previous node a final node w is reached after all other nodes have been considered; w needs to be connected to k other nodes as well. However, only $l-1$ of those can be part of the construction before since otherwise more than l nodes would be part of cycles, which yields a contradiction to the assumption. □

We are now ready to formulate lower and upper bounds for summary statistics of the number of nodes in weakly and strongly CCs of the k-NNG of Euclidean graphs.

Theorem 2.6. *For the number of nodes in the weakly connected components of the k-NNG G_k^u of any Euclidean graph with $\frac{n}{3} \geq k+1$, the following holds:*

1. *The minimum and maximum values of the mean are $\frac{n}{\lfloor n/(k+1) \rfloor}$ and n, respectively.*
2. *The minimum and maximum values of the median are $k+1$ and n, respectively.*
3. *The minimum and maximum values of the maximum are $k+1+\left\lceil \frac{(n \mod (k+1))}{\lfloor n/(k+1) \rfloor} \right\rceil$ and n, respectively.*
4. *The minimum and maximum values of the minimum are $k+1$ and n, respectively.*
5. *The minimum and maximum values of the span are 0 and $n-2(k+1)$, respectively.*

Proof. The upper bounds are trivial: all statistics except for the span take their maximal possible value n if there is just one single weakly CC containing all nodes; this is possible as shown in Theorem 2.2. The span takes its minimum, zero, in this case. As shown in Theorem 2.3 the minimal possible number of nodes in a weakly connected component in G_k^u is $k+1$. Thus, $k+1$ is as well the minimal possible value for the median if there are cases with more than half of the components having $k+1$ nodes. This is satisfied if two components have $k+1$ nodes and a third contains all remaining nodes. Note that for $k+1 > \frac{n}{3}$ this is not possible since then only two strongly components can exist as a weakly CC has at least $k+1$ nodes. The mean number of nodes in a weakly CC is smallest if the nodes are equally distributed between the highest possible number of weakly connected components as described in Theorem 2.3 and therefore is $\frac{n}{\lfloor n/(k+1) \rfloor}$. Like the median the maximal number of nodes in a component cannot be smaller than $k+1$ as well and if n is not divisible by $k+1$, it is smallest if the remaining nodes are evenly distributed between all components.

Eventually, the span of the number of nodes takes its maximal value if there are only two components, one containing the minimal number of nodes possible and one with all other nodes. Since the smallest number of nodes in a component is $k+1$ the maximal span is $n-2(k+1)$. □

Theorem 2.7. *For the number of nodes in the strongly connected components of the k-NNG G_k^d of any Euclidean graph with $n-2 > k$, the following holds:*

1. *The minimum and maximum values of the mean are $\frac{n}{n-k}$ and n, respectively.*
2. *The minimum and maximum values of the median are 1 and n, respectively.*
3. *The minimum and maximum values of the maximum are $k+1$ and n, respectively.*
4. *The minimum and maximum values of the minimum are 1 and n, respectively.*
5. *The minimum and maximum values of the span are 0 and $n-2$, respectively.*

Proof. Like in the case of weakly CCs (see Theorem 2.7) all statistics except for the span are maximal if there is only one single component containing all nodes. This is possible due to Theorem 2.2. As a consequence, the span of the number of nodes in the strongly CCs is minimal and zero in this case. As discussed in Theorem 2.5 the minimal possible number of nodes in a strongly connected component is one. Hence, one is the minimal possible value for the median if there are cases with more than half of the components having one node. This is satisfied if two components have one node and a third contains $n-2$ nodes. The minimal mean value is reached if nodes are distributed between the maximum number of strongly connected components as described in Theorem 2.5. Therefore the maximum mean value is $\frac{n}{n-k}$. The maximal number of nodes in a strongly CC is minimal in the same case.

As argued in Theorem 2.5 there is always at least one component with $k+1$ nodes; this is a lower bound for the maximal number of nodes in a single component.

The span is maximal if there are only two components, one containing the minimal number of nodes possible and one with all other nodes. Since the smallest number of nodes in a component is one the maximal span is $n-2$. □

2.3 Minimum Spanning Tree Based Features

Another crucial group of TSP features relies on a minimum spanning tree of the input instance.[3] Given a MST, this feature group consists of minimum, maximum, mean, median and the span of

[3] Note that minimum spanning trees are not necessarily unique.

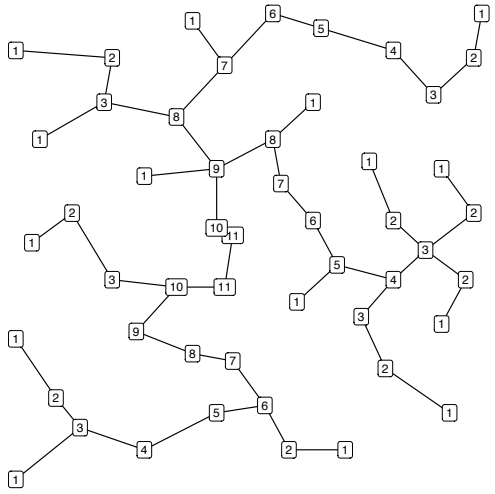


Figure 3: MST node depth calculation on eil51.

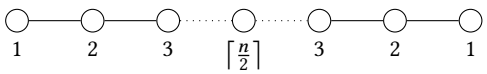


Figure 4: An n-vertex path of odd length. Each node is labeled with its depth.

the node depth in a MST of the source graph.[4]. The algorithmic implementation of the node depth calculation is as follows: an initial depth value $D = 1$ is initialized. Next, the algorithm removes leaf nodes recursively labelling them with D and increasing D after each iteration by one. The process is repeated until all nodes are labelled. Figure 3 visualizes the process on TSPLIB instance eil51.

Lemma 2.8. *Among all trees with n nodes, the n-vertex path has the maximal sum of depths.*

Proof. Note that the depth calculation does an "onion-peeling" by iteratively removing leaf nodes. For a tree, in every iteration at least two leaf nodes are removed. Now on a n-vertex path the algorithm performs exactly $\lceil n/2 \rceil$ iterations while on any other graph the number of iterations is upper bounded by $\lceil n/2 \rceil$. □

Theorem 2.9. *For the n-vertex path we have:*

(1) The mean depth equals $\frac{n}{4} + \frac{1}{2}$ for even n and $\frac{\lceil n/2 \rceil^2}{n}$ for odd n.
(2) The median depth is $\frac{n}{4} + \frac{1}{2}$ if $n \mod 4 = 0$ and $\lceil \frac{n}{4} \rceil$ otherwise.
(3) The maximum depth is $\lceil \frac{n}{2} \rceil$ and the span $\lceil \frac{n}{2} \rceil - 1$.

Proof. Ad (1): We distinguish n even and n odd. For even n we see that each depth between 1 and $n/2$ is counted twice (see Figure 4). Hence, the mean depth is given by

$$\frac{1}{n}\sum_{i=1}^{n/2} 2i = \frac{2}{n}\sum_{i=1}^{n/2} i = \frac{2}{n} \cdot \frac{(n/2)(n/2+1)}{2} = \frac{n}{4} + \frac{1}{2},$$

[4]In fact, this feature group consists of further features that rely on edge weight/distance statistics in the MST. For this study though, we do not consider any distance-based features.

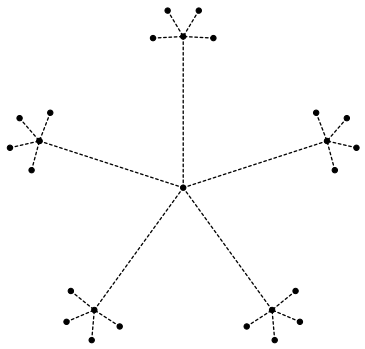


Figure 5: The MST depicted by the dashed lines corresponding to the case of minimal mean depth.

where we used the formula for the arithmetic sum $\sum_{k=1}^{n} k = n(n+1)/2$. For odd n we obtain

$$\frac{1}{n}\left(\sum_{i=1}^{\lfloor n/2 \rfloor} 2i + \left\lfloor \frac{n}{2} + 1 \right\rfloor\right) = \frac{\lfloor n/2 \rfloor(\lfloor n/2 \rfloor + 1) + (\lfloor n/2 \rfloor + 1)}{n}$$
$$= \frac{(\lfloor n/2 + 1 \rfloor)^2}{n} = \frac{\lceil n/2 \rceil^2}{n}.$$

Ad (2): Let $x_{(i)}$ denote the i-th order statistic of the depth values. Note that all but one depth value occur twice for an n-vertex path. Therefore, $x_{(\lceil n/2 \rceil)} = \lceil n/4 \rceil$. Hence, the median value is $\lceil n/4 \rceil$ for both odd and even n. The case where n is a multiple of 4 requires special attention. Here, the values $x_{(\lceil n/2 \rceil)}$ and $x_{(\lceil n/2 \rceil + 1)}$ differ by one. Therefore,

$$\frac{x_{(\lceil n/2 \rceil)} + x_{(\lceil n/2 \rceil + 1)}}{2} = \frac{n/4 + n/4 + 1}{2} = \frac{n}{4} + \frac{1}{2}.$$

Ad (3): The values for maximum depth and span are obvious. □

The structure of a MST with minimal depth is not trivial since a node within a MST of a Euclidean graph in the two dimensional space cannot have a degree greater than six as shown in [26]. In the following, we will first describe the construction of a graph with a variable number of nodes n that we will then show to have a MST of minimal possible depth.

In this regard, consider a graph $G = (V, E)$ that follows the fractal structure depicted in Figure 5 and – "zoomed in" for more details – in Figure 6. All nodes are placed around an *initial* node $I \in V$ with five nodes $P_i, 1 \leq i \leq 5$ being placed on the vertexes of a regular *pentagon* and $\forall v_j\ d(I, P_i) = r_1 < d(I, v_j),\ v_j \in V,\ v_j \neq P_i,\ 1 \leq i \leq 5$. Around the nodes P_i four nodes $C_{3,i,k},\ 1 \leq k \leq 4$ are placed. Note that the index of the nodes $C_{l\,i\,k}$ consists of three parts that locate the node in the fractal structure. The first part l states the level of the fractal structure the node belongs to, i identifies the node P_i that the considered node is allocated to and k counts the nodes with same l and i. For the four nodes around every P_i it holds that $d(C_{3,i,k},\ C_{3,i,l}) = d_3 > d(P_i, C_{3,i,l}) = r_2,\ 1 \leq k \leq 4,\ l \neq k,\ 1 \leq l \leq 4$. This ensures that $\forall l,\ \forall k,\ \forall f,\ \forall t\ \ d(C_{l,i,k}, C_{f,i+1,t}) < r_1$ where the radius r_2 is set to be $(d(P_i, P_{i+1}) - r_1)/3$. $C_{3,i,1}$ is placed to the left of P_i between the circle c_1 with radius r_1 around I and the tangent t of c_1 on P_i with a distance to t of one third of the distance

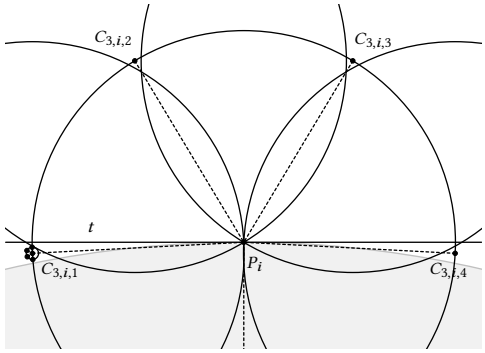


Figure 6: The repetitive pattern if more nodes of further levels are added. Dashed lines are the MST-edges and circles represent the area within that no other node can be placed without a change of the edge of the MST. The grey circle part is from the circle around the initial node I. $C_{3,i,4}$ and $C_{3,i,1}$ lie between the border of this circle part and the tangent t at P_i. Around $C_{3,i,1}$ nodes of the next level are depicted.

between the intersections of t and and c_1 with the circle c_2 with radius r_2 around P_i. The same applies for $C_{3,\,i,\,4}$ in a mirrored way. This construction pattern is repeated for nodes of greater levels in the fractal structure. Those nodes $C_{l,\,i,\,k}$ are added analogously but instead of I now $C_{l-2,\,i,\,\lceil \frac{k}{16} \rceil}$ is considered and instead of P_i now $C_{l-1,\,i,\,\lceil \frac{k}{4} \rceil}$ is considered. The MST $T = (V, E_T)$ of this graph contains the edges of G that connect the initial node with every P_i and those edges that connect every node with its four surrounding nodes of the next level. This is because G is constructed in such a way that those edges are the shortest of the node of the lower level to any other node.

Theorem 2.10. *The mean depth of a MST with $n > 7$ nodes is minimal if its structure corresponds to the structure of graph T.*

Proof. a MST with a smaller depth would either have more than five nodes around the initial node or on average more than four nodes around every succeeding node. As stated before the maximal number of nodes that can be connected to one single node within a MST of a Euclidean graph is six for the case of the two dimensional space [26]. If six nodes are placed around I the eighth point $C_{3,1,1}$ cannot lie between t and c_1 since then $d(P_5, I) > d(P_5, C_{3,1,1}) \vee d(P_2, I) > d(P_2, C_{3,1,1})$ and hence, $(I, P_5) \notin E_T \vee (I, P_2) \notin E_T$. Therefore, only two nodes of the next level can be connected to every P_i in this case. Thus, the MST of this case has a smaller depth for $n = 7$, an equal depth for $(n < 7) \vee (7 < n < 10)$ and a smaller depth for $n > 9$. If a MST would have on average more than four connected succeeding nodes around every node other than the initial node this means that some nodes would have five or six connected succeeding nodes. Six connected succeeding nodes are not possible since then no preceding node could be connected to the considered node. If a node is connected to five succeeding nodes the preceding node has to lie at the last free point for the case of six nodes around one node. Thus, the preceding nodes of the considered node could only have two preceding nodes as in the case for the six nodes around I yielding an average of less than four preceding nodes per node. □

With this, the minimal possible median depth of the MST is one since more than half of the nodes are leaves. The maximal depth in this case can be calculated by first approximating the greatest completed level L of this tree with $\lceil \log_4(\frac{n-1}{5}) \rceil + 1$. With this, the number of remaining nodes R that are not sufficient to complete the current level can be calculated by $n - (1 + \sum_{i=1}^{L-1} 5 \cdot 4^{i-1})$. If $R > 4^{L-1}$ the depth of the root node and therefore the maximal depth is $L + 1$ and, otherwise L. In both cases, the span is the max depth minus one. If R is negative, L was overestimated by 1 and first has to be corrected in order to compute the mean depth. Accordingly, R has to be recalculated as well. Second, the sum of all depths without R is computed by $L + \sum_{i=1}^{L-1} 5 \cdot 4^{i-1} \cdot (L - i)$. Then, the growth of the sum of all depths by adding R nodes can be calculated with a recursive function $f(r, l)$ with r being the number of nodes at level l that have a greater depth because of the remaining nodes R. If $l = 1$ the function evaluates to $r + \mathbb{1}_{r>1}$ where $\mathbb{1}_{r>1}$ is the indicator function which evaluates to 1 if the predicate $r > 1$ is true and to 0 otherwise, and else $r + f(\lceil \frac{r}{4} \rceil, l - 1)$. Finally, the mean of all depths can be calculated by

$$\frac{1}{n}\left(L + \sum_{i=1}^{L-1} 5 \cdot 3^{i-1} \cdot (L - i) + f(R, L)\right).$$

After deriving the theory of normalizing TSP features, we proceed with an empirical analysis and testing of our outcome in the following sections. With this we demonstrate the advantages of normalized TSP features.

3 INSTANCES AND ALGORITHMS

In this section, a brief overview of the used TSP instances, considered TSP solvers and validation of the solvers' runtimes is given. The full set of considered instances is the union of three sets from the literature enhanced by new, generated instances.

The first set was already used in Kerschke at al. [14] and is referred to as ECJ2018 in the following. It consists of a collection of instances from different sources, including artificially generated instances such as random uniform Euclidean (RUE) instances, strongly clustered instances (Netgen), and hybrids "in-between" resulting from "morphing" each one RUE and Netgen instance (Morphed); for details on the morphing process we refer the interested reader to [18]. In addition, the set contains real-world instances from the well-known TSPLIB [24] benchmark (TSPLIB), instances from the very large scale integration (VLSI) application area and instances considering subsets of real-world cities (National). Figure 7 gives an exemplary impression of the structural differences between the instance types. In total, there are 1 844 TSP instances in this set. Next, we also used the instance set that was proposed in [5] consisting of 450 instances in total. The number of cities for each instance is $n \in \{500, 1\,000, 2\,000\}$. These instances were generated in an evolutionary process using sophisticated mutation operators with the goal to evolve instances which show severe performance differences between state-of-the-art solvers EAX and LKH (see below). We refer to this set as FOGA2019. The third set of instances is taken from [28]. It consists of 1 000 instances with a size of $n = 1\,000$ cities evolved by the method proposed in [5]. 500

Table 1: Overview of all instance subsets including the PAR10 and PQR10 scores for the three considered solvers EAX, LKH and Concorde (best performing solvers are highlighted in **bold** while second bests are underlined).

Set	Num. of Cities				Num. of	PAR10			PQR10		
	Min.	Median	Mean	Max.	Instances	EAX	LKH	Concorde	EAX	LKH	Concorde
ECJ2018 [14]											
RUE	500	1 250	1 250	2 000	600	**16.53**	163.13	1 319.92	**12.82**	150.08	1 284.95
VLSI	662	1 412	1 311	1 973	18	**5.46**	59.85	6 544.21	**4.68**	41.91	6 230.43
TSPLIB	574	1 291	1 195	1 889	21	**1 726.82**	1 794.12	11 554.98	**1 722.84**	1 764.66	11 849.54
National	634	866	1 028	1 979	5	**4.59**	19.63	513.60	**3.55**	12.41	471.38
Netgen	500	1 250	1 250	2 000	600	**10.16**	669.33	253.84	**8.49**	574.24	248.21
Morphed	500	1 250	1 250	2 000	600	**13.88**	678.12	481.73	**10.70**	583.05	466.47
FOGA2019 [5]											
All	500	1 000	1 167	2 000	450	**10.89**	421.90	798.92	**8.61**	410.19	779.91
Evolved-Simple Instances											
500-EAX	500	500	500	500	1 000	**1.25**	5 524.22	43.37	**1.12**	5 127.05	42.22
500-LKH	500	500	500	500	1 000	110.23	**0.89**	62.39	80.71	**0.73**	60.97
1 000-EAX	1 000	1 000	1 000	1 000	500	**4.79**	3 940.62	364.46	**3.98**	3 306.38	357.22
1 000-LKH	1 000	1 000	1 000	1 000	500	146.28	**44.29**	385.05	101.38	**44.04**	376.17
1 500-EAX	1 500	1 500	1 500	1 500	500	**12.65**	2 818.57	706.16	**10.14**	2 504.68	686.15
1 500-LKH	1 500	1 500	1 500	1 500	500	208.57	**14.70**	667.13	131.27	**10.79**	651.24
2 000-EAX	2 000	2 000	2 000	2 000	500	**25.35**	2 877.67	1 399.64	**19.62**	2 458.15	1 367.76
2 000-LKH	2 000	2 000	2 000	2 000	500	258.61	**42.05**	1 263.46	155.46	**25.44**	1 233.53
Evolved-Sophisticated Instances											
500-EAX	500	500	500	500	1 000	**1.01**	7 987.63	23.15	**0.95**	7 952.74	22.42
500-LKH	500	500	500	500	1 000	62.17	**1.32**	49.80	46.53	**1.08**	48.67
1 000-EAX [28]	1 000	1 000	1 000	1 000	500	**3.92**	7 128.84	207.23	**3.40**	6 994.79	202.97
1 000-LKH [28]	1 000	1 000	1 000	1 000	500	138.25	**7.73**	281.47	88.75	**5.49**	273.51
1 500-EAX	1 500	1 500	1 500	1 500	500	**9.49**	6 192.82	534.80	**7.79**	5 422.83	516.74
1 500-LKH	1 500	1 500	1 500	1 500	500	286.27	**16.54**	615.87	165.50	**12.91**	597.14
2 000-EAX	2 000	2 000	2 000	2 000	500	**19.17**	4 480.48	1 196.17	**15.25**	4 064.28	1 161.64
2 000-LKH	2 000	2 000	2 000	2 000	500	225.05	**38.14**	1 119.17	135.98	**22.73**	1 091.05

of these instances are easy to solve for EAX and the other 500 are easy to solve for LKH. In addition, we generated another large set of evolved instances with $n \in \{500, 1\,500, 2\,000\}$. In combination with the instances from [28], this set is referred to as evolved instances. We extended the set of [28] with additional evolved instances to increase the diversity of the artificially generated instance set. Hence, we considered not only different numbers of cities but also the simple mutation operators [5] which have noticeable different characteristics in comparison to the sophisticated mutation operators. For an overview of all considered instance sets see Table 1 and Figure 7 for some visualizations of exemplary considered instances.

We are interested in improving the state of the art in TSP-solving by means of per-instance algorithm selection. Therefore, our algorithm portfolio consists of the following three algorithms: *edge assembly crossover* (EAX) [19], *Lin-Kernighan-Helsgaun heuristic* (LKH) [9, 17], and the exact solver Concorde [1]. EAX is a genetic algorithm with a smart crossover operator (termed EAX as well) and entropy-based diversity preservation to prevent premature convergence of the population. LKH is Keld Helsgaun's implementation of a highly effective local-search algorithm based on k-OPT edge exchanges. Both solvers, in their respective restart versions [8], pose the state-of-the-art in inexact TSP-solving [14, 16].[5] Concorde on the other hand is an exact solver based on a highly developed ILP-based branch-and-cut procedure involving cutting-plane methods; Concorde is frequently capable of solving instances with thousands of nodes to optimality in reasonable time which is astonishing given the problems' NP-hardness.

We solve all TSP instances and benchmark their performance against each other. Each solver was executed ten times on each instance and the needed runtime was recorded. We then calculated the *penalized average runtime* (PAR10; [2]) and the less outlier-sensitive *penalized quantile runtime* (PQR10; [12]) scores for each solver and instance. Both measures aggregate the runtime over the ten runs. When a solver was not able to find the optimum within the give time-budget, the solver receives ten times the cutoff-time as penalty. In contrast to PAR10, which computes the arithmetic mean of the penalized runtimes, PQR10 is based on the p-quantile, with $p = 0.5$ (i.e., the median) in our case. We used a cutoff-time of 3 600 seconds for EAX and LKH while Concorde had an unlimited runtime. The reason is that EAX and LKH are heuristic solvers. Both do not guarantee to find the optimal solution to a given TSP instance. Instead, they may get stuck in a local optimum and even

[5] The restart-version triggers a restart once the algorithm runs into its internal stopping conditions and the given time budget is not yet depleted.

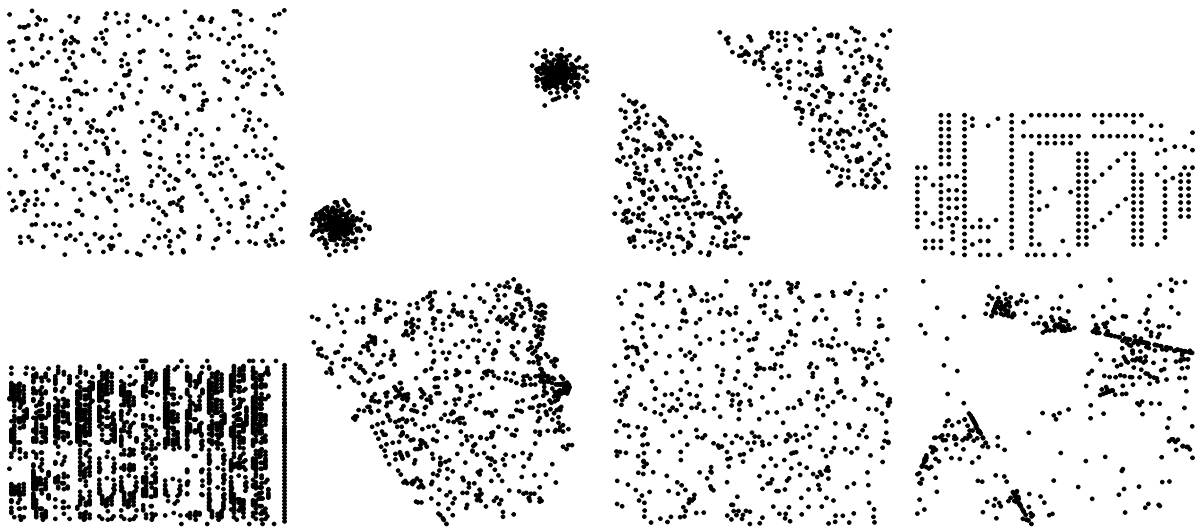


Figure 7: Exemplary instances from the considered instance set. From left to right row by row: RUE, Netgen (two dense clusters), Morphed (combination of the two preceding instances), TSPLIB, VLSI, National, and evolved (with simple mutation) as well as evolved (with sophisticated mutation).

an unlimited time-budget will not prevent that. In contrast to EAX and LKH, Concorde is an exact solver — it will find the optimum within an unlimited period of time. The PAR10 and PQR10 scores for the different solvers and instance sets can be found in Table 1. It becomes evident, that for all instance sets, EAX and LKH can be considered to be competitive to each other. In fact, 51.6% of all instances are EAX-easy instances — meaning, that EAX is the quickest solver to find the optimum. In addition, 48.3% of all instances are LKH-easy while only 0.04% of all instances are Concorde-easy. We found that Concorde is not sufficiently competitive to EAX and LKH and is, therefore, not considered in our case study.

4 EXPLORATORY DATA ANALYSIS

Next, we conduct an exploratory data analysis (EDA) on the instance groups introduced in Section 3 to assess the feature normalization effects visually. First, we evaluate how the normalization affects the TSP features themselves. Then, we deepen the analysis by assessing how the normalization affects the correlation of the features (a) to other features and (b) to the performances of EAX and LKH.

4.1 Normalized vs. Unnormalized Features

We first compare scatterplots of normalized and unnormalized features respectively. Some exemplary, yet representative results on the FOGA2019 instance set can be found in Figure 8. Here, the maximal number of nodes in strongly CCs in the 3-NNG is shown on the x-axis and the number of strongly CCs in a 3-NNG is plotted on the y-axis. The plot on the left shows the unnormalized features, the one on the right the respective normalized counterparts.

Considering the node coloring in the unnormalized plot, the different instance sizes ranging from 500 to 2 000 nodes are distinguishable. We observe that the number of strongly CCs increases with the instance size. Thus, the clusters in the plot represent the instance size. In contrast to that, in the normalized plot, the instance sizes are not the discriminating factor. The distinctively colored clusters that represent different instance sizes overlap clearly. Considering the other instance groups – ECJ2018 and evolved instances – the respective scatterplots show the same pattern. These results support the expected effects of the normalization. Concededly, when considering the elementary normalization by Pihera & Musliu [22], the results are visually similar. Nevertheless, the normalization conducted here is much more precise and mathematically well-founded. It therefore bears more valuable theoretical insights into the effects of TSP feature normalization.

Admittedly, there still exist a few other plots of normalized features in which the instance size can still be distinguished. Here, the instances with a higher number of nodes have lower values than instances with fewer nodes. Nevertheless, this phenomenon has a reasonable mathematical explanation: our normalization considers the lower and upper bounds of the feature values, f_{min} and f_{max}. For some features, e.g., the maximal depth within a MST, the upper bound grows linearly while the lower bound grows logarithmically with n. Thus, if the feature value on average grows logarithmically with n as well, the normalized value approaches zero for increasing n.

Furthermore, we explored the effect of the normalization when considering multiple features. To this end, we adopt autoencoders, a modern dimensionality reduction method. Autoencoders are feed-forward deep neural networks that learn data representations in an unsupervised manner. Autoencoders are composed of encoder and decoder networks as well as a latent space in-between. The encoder uses the usually high-dimensional input data and transforms it into a low-dimensional representation in the latent space. The decoder then uses these low-dimensional encodings to retrieve the original

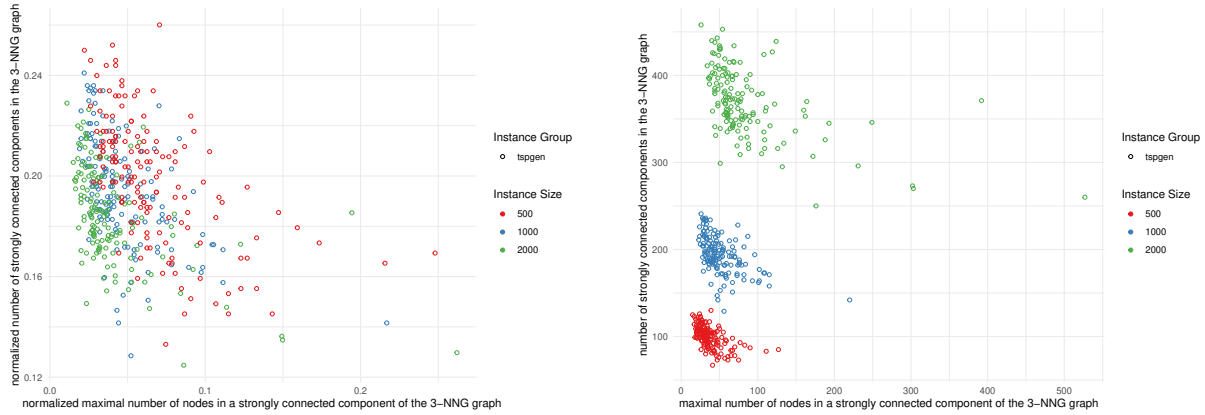


Figure 8: Scatterplots of two 3-NNG graph features displaying the effect of the TSP feature normalization based on the FOGA2019 instance set. On the left-hand side, the unnormalized features can be seen, and on the right-hand side their normalized counterparts. The normalization worked as expected since the instance sizes cannot be distinguished clearly anymore.

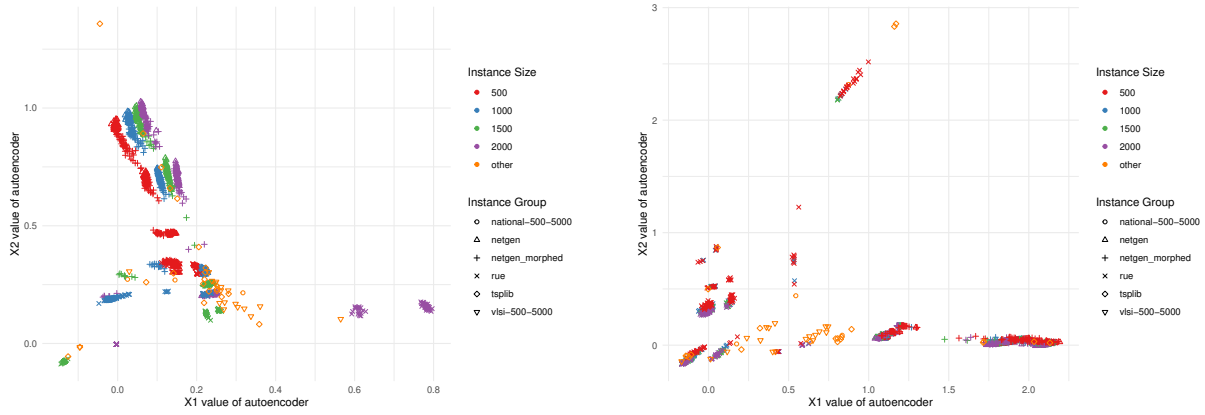


Figure 9: Result of the reduction of 39 MST- and NNG-features to the two-dimensional space by using autoencoders. The plots show the two-dimensional representations for the unnormalized features (left) and the normalized features (right).

high-dimensional data. Overall, the goal is to minimize the autoencoder's reconstruction error to recover an exact low-dimensional representation of the data. The effectiveness of autoencoders was first shown in [10]. We also considered other prominent dimensionality reduction methods like principal component analysis (PCA), multidimensional scaling (MDS), and t-distributed stochastic neighbor embedding (t-SNE). However, we decided against these classical approaches since they have problems capturing local relationships as was shown in [27] and/or perform worse than autoencoder (see, e.g., [10]).

As the autoencoders were initialized with random weights, the resulting mappings vary between autoencoders, even though they were trained using the same data set. Hence, we trained 30 networks for the normalized data and 30 for the unnormalized data and found that they have a very similar performance and revealed the same pattern. The performance was assessed using trustworthiness and continuity as proposed in [30]. *Trustworthiness* measures to what extent the z-closest neighbors in the embedded space are also close in the original space, and *continuity* to what extent the z-closest neighbors in the original space are also close in the embedded space. Hence, trustworthiness penalizes close neighbors in the embedded

Table 2: The aggregated trustworthiness and continuity scores for the z-closest neighbors of the 30 independently trained autoencoders. Performing a one-sided Wilcoxon test yields that the autoencoder on normalized features have a significantly higher continuity (p-value $\ll 0.05$) whereas the results for trustworthiness are insignificant (p-value $= 1$).

	Unnormalized				Normalized			
	trustworthiness		continuity		trustworthiness		continuity	
z	median	iqr	median	iqr	median	iqr	median	iqr
1	0.969	0.007	0.988	0.005	0.963	0.012	0.993	0.002
5	0.969	0.006	0.985	0.006	0.963	0.012	0.989	0.002
12	0.969	0.006	0.982	0.006	0.961	0.012	0.987	0.003

space that are not close in the original space and continuity penalizes neighbors in the original space that are not close in the embedded data. Both values range from 0 to 1 with values closer to 1 being better. The results of our experiments can be seen in Table 2 for both the unnormalized (left part of the table) and the normalized features (right part). As it is not clearly visible which of the two groups of autoencoders encodes the features better, we performed a one-sided Wilcoxon test. The test yields that the continuity scores are significantly higher for the autoencoders that are based on the normalized features (compared to the ones based on the unnormalized features). At the same time, the test results were insignificant for the trustworthiness. It is thus evident that the normalization could substantially improve the continuity but fails to improve the trustworthiness. This implies that autoencoders based on the normalized features can preserve more of the original local structure than those based on the unnormalized features. Figure 9 shows the result of a representative autoencoder mapping. It reduced the set of 39 NNG- and MST-features to the two-dimensional space represented by X_1 and X_2. Note that the underlying autoencoders have been trained on all instance sets listed in Table 2, however, for better readability we only visualized mappings for an exemplary subset of the instances, namely ECJ2018.

In the unnormalized plot, the different instance sizes can be discriminated clearly, indicated by the coloring of the instances. In the normalized plot, the instances of different sizes are not separable anymore. This is in line with our observations on feature pairs (see Figure 8) and indicates that the normalization has the desired effect.

Next, we analyzed the correlation of the features to each other. Heatmaps of the correlation coefficient for the normalized and unnormalized features on the FOGA2019 set can be seen in Figure 10. High positive correlations are colored in red, whereas blue color tones indicate a negative correlation.

As can be seen in the plot on the left-hand side, the correlation is diverse among the set of features. Noticeably, there are light areas in the normalized feature correlation plot, which indicate a correlation close to zero. Nevertheless, there exist groups of highly positively correlated features. In particular, a red block in the middle of the plot, consisting of NNG-features related to the minimal, maximal, and span features of the strongly and weakly CCs, stands out. These features are also positively correlated with the features related to the depth of the respective MST.

In contrast to that, when considering the normalized heatmap in Figure 10, the overall impression of the correlation changed notably. Here, the groups of highly correlated features are significantly smaller. The large block of highly correlated features is not visible anymore. The normalization revealed that the correlation between these features is not as high as it seemed. Additionally, when considering the relation of these features to the MST features, the correlation flipped in the negative space. A similar phenomenon can be observed when considering the features calculated on the evolved instance set. Also, for the ECJ2018 instances, the correlation decreased or shifted notably in the negatively correlated space, although a large group of positively correlated features remains. When considering these findings, eventually, researchers might have overestimated the intensity of the feature's interrelation in the past. This insight is interesting concerning the algorithm selection study, which we will conduct in Section 5.

4.2 The Impact of Feature Normalization on the Solver Performance

We made a similar observation when we considered the correlation of the normalized and unnormalized features to the performance of EAX and LKH. Notice that in Figure 11, the points depict unnormalized or normalized features and not the different instances anymore. The plot reflects the correlation of the features with the PAR10 value of LKH on the abscissa (x-axis) and the PAR10 value of EAX on the ordinate (y-axis). It was created based on the evolved instances.

Ultimately, the goal of the feature normalization is to gain insights for an algorithm selection study as we will conduct in Section 5. Having this in mind, the points in Figure 11 representing the features should ideally reside in the second (top left) or fourth quadrant (bottom right) of the plot. Here, the correlation of the features to the performance of one algorithm is positive and negative to the other algorithm. These features may facilitate the algorithm selection process. To better visually separate the favorable from the unfavorable quadrants in Figure 11, we colored the favorable ones in green and the less favorable ones in red.

In the left plot of Figure 11, less than 50% of the features are located in one of the favored quadrants. Most features reside in the upper right quadrant, indicating a positive correlation of the features to the performance of both algorithms.

On the right, we plotted blue arrows in the normalized plot, indicating the shift from the former position in the unnormalized plot. Like the correlation of the features themselves depicted in Figure 10, the correlation of the features to the algorithm performance shifted in the uncorrelated or negatively correlated space. The features related to the span, the mean, and the maximal value of the depth of the MST moved furthest, from the positively correlated space in the left part of Figure 11 to the negatively correlated space in the right part of Figure 11.

Unaffected by the normalization is the correlation of six features, which can be seen in the middle of the plot. For example, three of them are the minimal number of nodes in a strongly CC in a 5-NNG, the span of the number of weakly components in a 5-NNG, and the span of the number of weakly components in a 7-NNG. That is the case since many unstructured instances consist of a single weak component per definition. Similarly, the minimal number of nodes in a strongly CC in a 5-NNG is equal to one. Therefore, the correlation did not change with the normalization for these three features. The feature values are constantly zero or one, respectively.

It is also worth noticing that many features from the upper right quadrant in the unnormalized plot moved in the favorable upper left quadrant in the normalized plot. The same is true for the other green-colored quadrant in the normalized plot. Now, most of the features are located in these two quadrants. This result is a desirable effect of the normalization since it indicates that some features are more suitable for distinguishing algorithm performance than initially thought. These effects are also observable for ECJ2018 or FOGA2019 instances. Notice also that the features – independent of their normalization status – are slightly more correlated with the performance of EAX than with the performance of LKH. This might also explain why AS models – e.g., the ones in Section 5 – tend to select EAX more often than LKH.

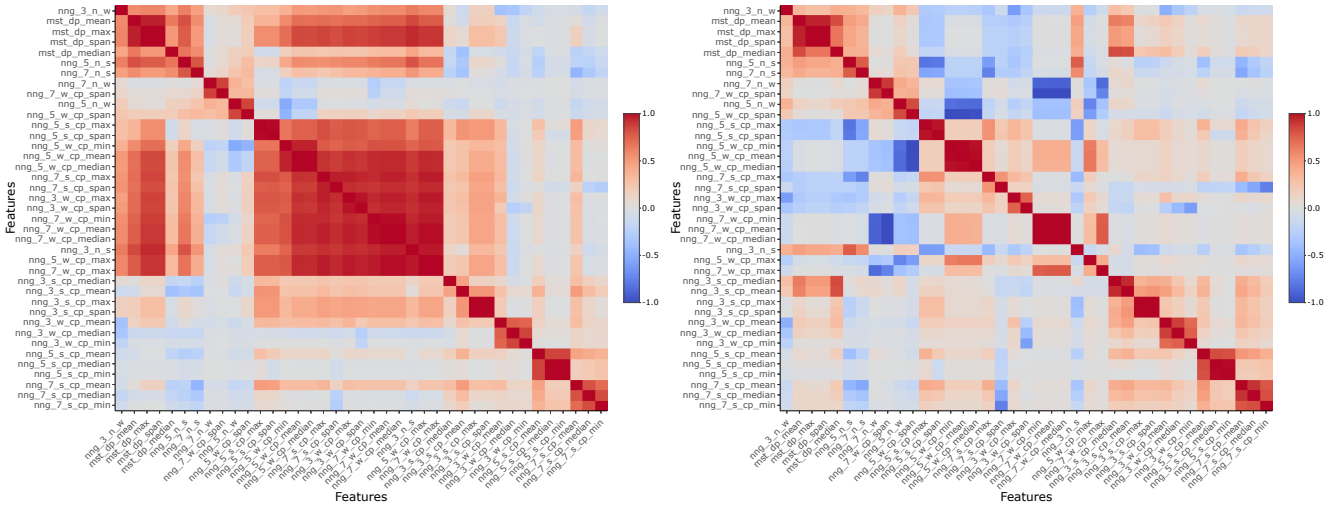


Figure 10: Feature correlation heatmaps (left: unnormalized, right: normalized).

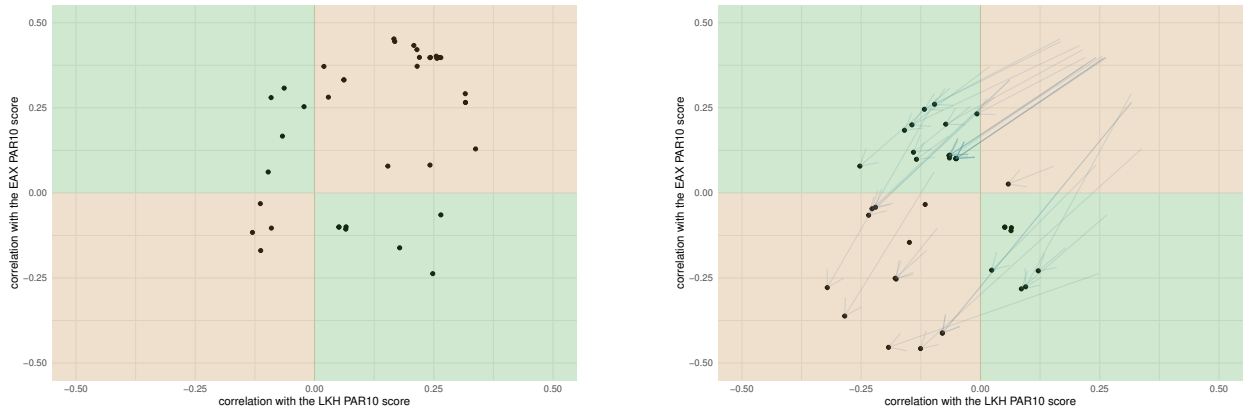


Figure 11: Correlation of LKH and EAX performances with the features. The two plots show the correlations of the algorithms' performances with the unnormalized features (left) and normalized features (right), respectively. Additionally, in the normalized feature plot, the arrows indicate the shift of the features that is caused by their normalization.

When considering the arrows' flows, we observe that for some features, the correlation to the performance of LKH decreased more, and for other features, the correlation to the performance of EAX. Thus, the normalization generates an overall arched movement of the arrows. However, this phenomenon is only observable for the evolved instances and neither for the ECJ2018 nor FOGA2019 instance sets. For the latter two instance sets, the arrows are arranged straight and indicate no difference in the decrease of the correlation for one of the algorithms. The evolved instances created by [5] and [28] were generated to be solved more easily by either EAX or LKH. The normalization reveals that this goal was achieved since most of the features calculated based on the instances are more correlated with one algorithm.

We can draw a few conclusions after conducting the EDA: first, the normalization effects are visible. As can be seen in Figure 8 and 9, the instance sizes are no longer the primary discriminating factor. Second, the correlation of the normalized features to other normalized features decreased notably in contrast to their unnormalized counterparts. Third, the correlation of the features to the performance of EAX and LKH changed notably. Although it decreased overall, it is revealed that most features are positively correlated with the performance of one algorithm and negatively with the performance of the other algorithm. These features might be more auxiliary for an ML-algorithm to predict the performance of both algorithms as estimated.

All instance sets, algorithm performances, and plots are available online in a dashboard for further review.[6] In the following, we will test the suitability and effects of the normalized features for constructing automated algorithm selection models.

[6]The dashboard can be found at https://tsp-features.shinyapps.io/normalization-eda/ and the corresponding implementation at https://github.com/jonathan-h1/TSP-EDA.

5 CASE STUDY: ALGORITHM SELECTION

As the exploratory data analysis (see Section 4) revealed some changes in terms of feature visualization – which were caused by our proposed normalization – we will in the following explore the potential of the normalized features for automated algorithm selection. Note, however, that the objective of this work is not a comprehensive benchmark study. Instead, the goal of this section is a proof-of-concept study that may provide a first indication about the suitability of the proposed normalization[7].

Within our study, we utilized the following (normalized) features:

- summary statistics (mean, median, maximum and span) of an instance's MST depth (see Section 2.3),
- the number of weakly/strongly connected components in the k-NNG of a TSP instance (see Section 2.2), as well as
- summary statistics (minimum, mean, median, maximum and span) of the number of cities across the weakly/strongly connected components of an instance's k-NNG (see Section 2.2).

The NNG-features were based on k-NNGs with $k = \{3, 5, 7\}$ following the parameterization proposed in [22].[8] Hence, our data contained a total of 40 features. We then had to discard the normalized (unnormalized) feature `nng_3_strong_components_min` – i.e., the minimum number of cities in the strongly connected component of a 3-NNG – from our data, as it was constantly zero (one) across all considered instances and thus did not provide any information for our candidate selectors. Instead, we occasionally also considered the instance size as a feature as it might nonetheless be useful for discriminating the different TSP instances.

In line with previous AS studies in the context of (inexact) TSP solving, our experiments are based on the 1 844 ECJ2018 instances (see Table 1 for details). We further focused on the state-of-the-art (restart-variants [8]) of the two heuristic TSP solvers EAX [20] and LKH [9]. As candidates for the AS models, we investigated random forests and support vector machines, both of which are known to be very powerful machine learning algorithms. To avoid undesirable side effects such as redundancy and partially high correlations among features (see Section 4) and/or noise within features, we also performed automated feature selection. For simplicity, we considered a greedy forward-backward selection strategy, which usually tends to find small but informative feature subsets in a very efficient manner. Note that we also considered the overhead of feature calculation. The costs for the considered (normalized) feature sets though are negligible.

In terms of the selectors' performance assessment, we used the PAR10 score [2]. For a simpler comparison of the performance values, it is recommended to compare the performance of the AS model with (a) the performance of the best standalone optimization algorithm – also known as *single-best solver* or SBS (here: EAX) – and (b) the best possible performance, which corresponds to an oracle-like prediction of the best optimization algorithm per instance (also known as *virtual-best solver* or VBS). To facilitate this comparison, we subsequently converted the selector's PAR10 score into a closed-gap-percentage, indicating the proportion of the gap between the SBS and VBS performances, which is closed by the AS model. Note that this is a common metric in various AS studies [2].

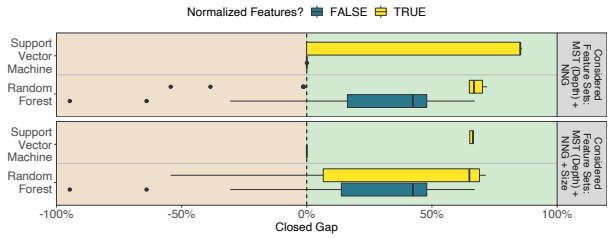


Figure 12: Comparison of the closed-gap-performance of different AS models, which are based on different feature sets. Models are trained with MST- and NNG-features (top) and in addition with the instance size (bottom).

In order to also take the stochasticity of the AS models into account, we repeated all our experiments 25 times. The resulting closed-gap-values are illustrated in Figure 12 by means of boxplots. As can be seen, the boxplots that correspond to the AS performances of the normalized features, tend to be better with respect to the median closed gap than the ones of the unnormalized features; this observation is confirmed by Wilcoxon-Mann-Whitney tests at a significance level of $\alpha = 0.05$ in all cases. Still, there appears to be quite large variability across the performances. However, SVM models in the case of unnormalized features are degenerated at a 0%-level (see Figure 12). This can be attributed to the models predicting EAX in most cases for instances that are actually easier for LKH (see Figure 14). We stress that the results are particularly positive if one takes into consideration the severely reduced number of features used in this study.

Due to the high volatility of the performances, we examine the most promising models in more detail below. Figure 13 visualizes, which features have been selected by the respective selector (support vector machine) in each of the 25 replications. In both cases, i.e., unnormalized (top) vs. normalized features (bottom), the selectors are mainly affected by two NNG-features, which oftentimes are complemented by up to four additional features. And although some initial patterns are visible within that feature importance heatmap, the patterns become more clear when looking at the corresponding (mis-)classification costs (see Figure 14). Quite noticeably, the SVM based on the unnormalized features does not learn a lot and instead, simply has a strong tendency towards constantly selecting EAX. On the other hand, the patterns within the misclassification heatmap for the normalized features reveal that depending on the replication, one of three outcomes had been learned:

- For roughly every third run (replications 2, 6, 8, 11, 14, 20 and 22), the selector also had a strong bias towards EAX (as indicated by the yellow boxes in the respective heatmap). When matching those performances with the corresponding selected features, one can observe that those runs are exactly the cases, for which the features `mst_depth-max` to `nng_3-strong_components_median` have *not* been selected by the model.

[7] The code for the experiments can be found at: https://github.com/jonathan-h1/FOGA21_TSP_Experiments.

[8] In their work, Pihera & Musliu also considered values of k which depend on n, e.g., $k = \sqrt{n}$. We neglect these versions since they are constant in most cases. The reason is that the number of weakly/strongly CCs in the k-NNG is monotonically decreasing as k approaches n. Hence, there is a $0 < k' < n$, such that for all $k \geq k'$ the number of connected components in the k-NNG is one.

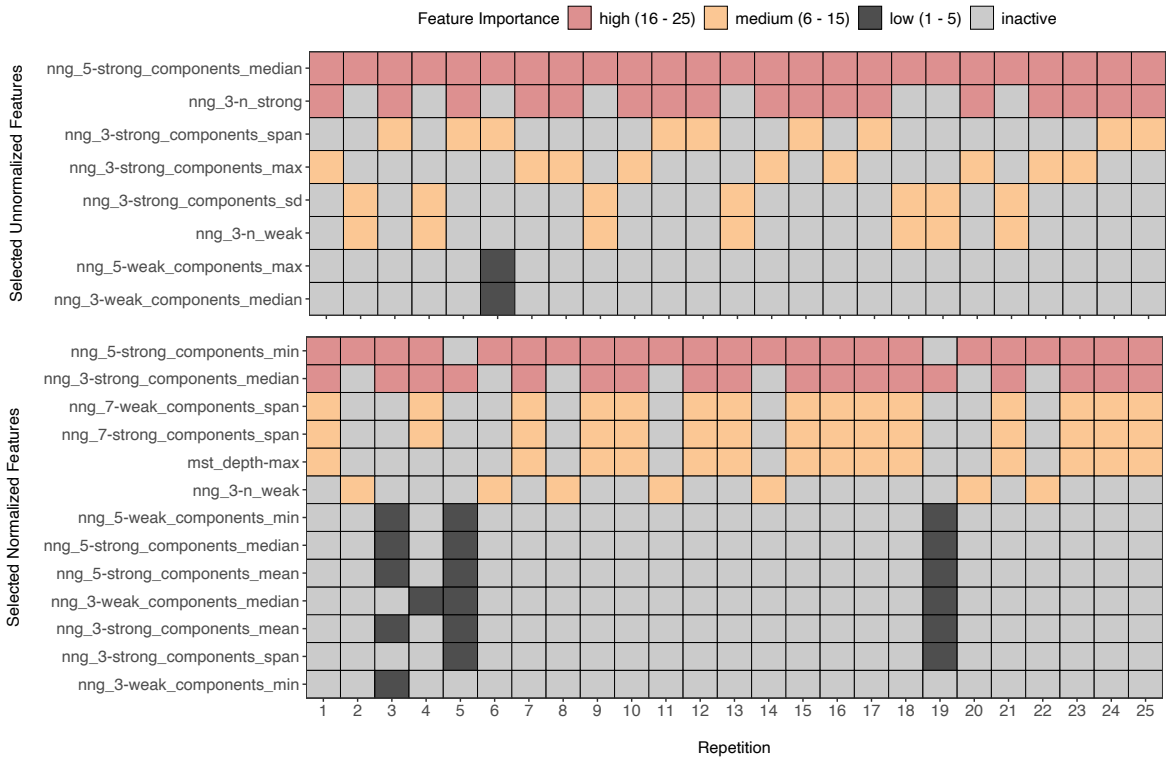


Figure 13: Features (rows) selected by the algorithm selector in the 25 replications (columns). The trained models are based on the unnormalized features (top), as well as the normalized features (bottom). The cell colors indicate how often a feature has been selected across all 25 replications.

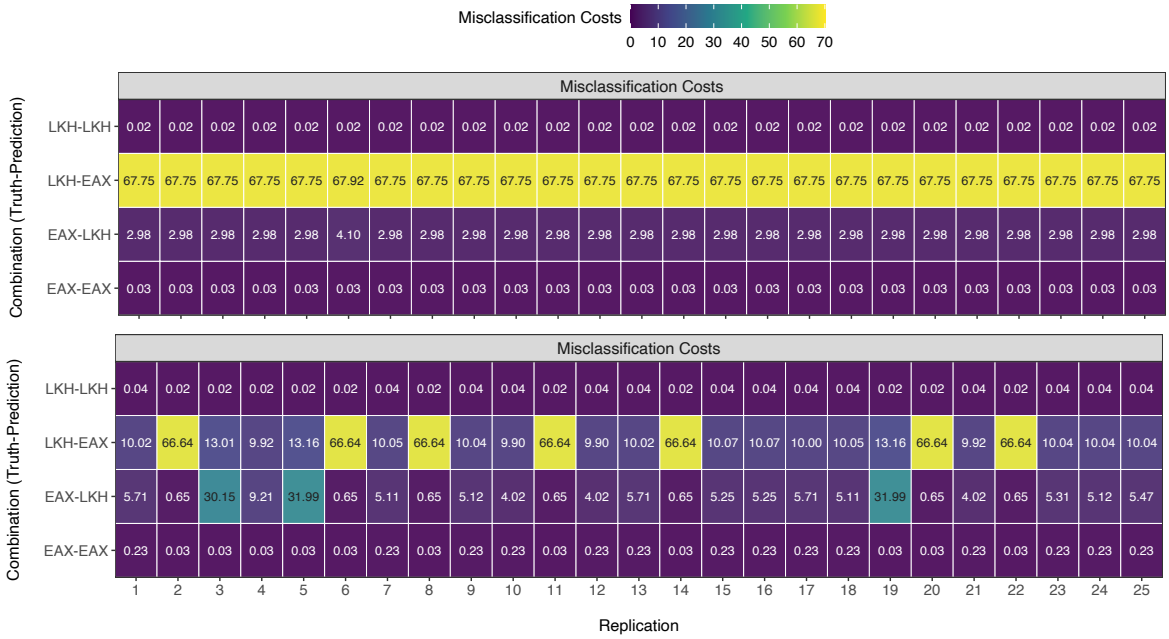


Figure 14: Misclassification costs of the algorithm selector across the 25 replications (columns). Results are shown for the selector based on the unnormalized (top) and normalized features (bottom) and distinguished by pair of true vs. predicted algorithm (rows). If the selector makes the correct prediction (EAX-EAX and LKH-LKH) the costs indicate the feature costs.

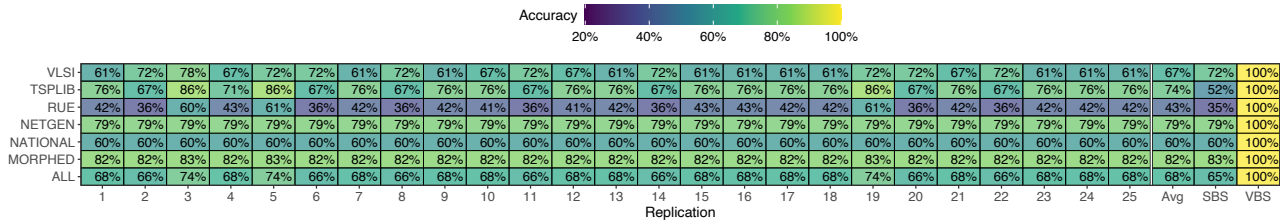


Figure 15: Accuracy of the SVM model on normalized features, separated by ECJ2018 instance subset (rows) and repetitions (columns). The final three columns show the model's accuracy averaged across all 25 runs, SBS and VBS values.

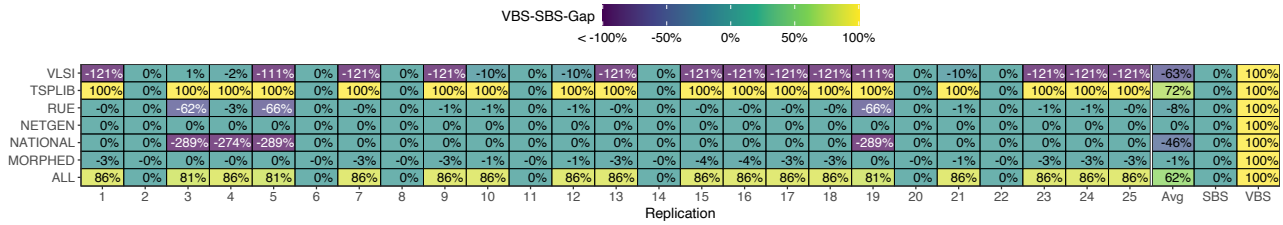


Figure 16: Percentage of the SBS-VBS-gap that was closed by our selector's prediction. Results are shown for the SVM model and separated by ECJ2018 instance subset (rows) and repetitions (columns). The final three columns show the model's averaged performance across all runs, as well as the SBS and VBS values.

- In three cases (replications 3, 5 and 19), the AS model had a bias towards LKH (as shown by the turquoise cells in the misclassification heatmap). Noticeably, those runs relate to the AS models, for which the majority of the bottom 7 features from Figure 13 have been selected (i.e., the ones indicated by black cells).
- For the remaining 15 out of 25 replications (i.e., 60% of all runs), the AS model performed very well. And in those cases, the five features shown at the top of the feature importance heatmap have been selected.

Now that we have a first indication of what our AS models have actually learned, we will investigate whether we can observe further patterns based on the underlying TSP subset (National, VLSI, RUE, etc.). For this, we compare the accuracies of the 25 separate AS models (one per replication) against the corresponding performances of the SBS in Figure 15. At first glance, all those values look rather similar (per row) and the differences appear to be rather minor. For instance, on the RUE instances, our AS model performs slightly better than EAX, whereas it performs worse for VLSI instances. The only subset, for which the difference in accuracy was noteworthy, are the instances from the TSPLIB.

This effect increases when looking at the SBS-VBS-gap-closure (see Figure 16). Aside from the 7 replications, for which our model was clearly biased towards EAX, our selector achieved optimal performance results on TSPLIB for the remaining 18 replications. The three replications for which our algorithm selector was biased towards LKH can also be seen in Figure 16. In those cases, our selector had severe problems on RUE instances. These observations are in line with [14] who trained their models on an extended set of (unnormalized) features. It is thus likely that the current features are not sufficient to capture the characteristics of these instances.

To summarize the proof-of-concept study: the proposed feature normalization has flashed its potential for AS studies. However, the high variance in the performance of the selectors also indicates that the proposed normalization on its own does not automatically guarantee a perfect AS model. In fact, we identified three different selectors, each of them specified by a particular subset of (normalized) TSP features. Further in-depth investigations of the three selectors found and their respective used features could ultimately lead to a better understanding of typical properties and challenges of TSP instances, and in particular how these affect the different optimization algorithms.

6 CONCLUSION

In the context of automated algorithm selection (AS), so-called instance features are numeric values that characterize problem instances of optimization problems; these features serve as the input variables for machine learning models linking features to solver performance results. Scalability of AS models across the number of nodes in the context of TSP so far was limited as instance feature levels are influenced by the instance size. Instance features were either not normalized at all or normalization was done imprecisely by plain division by the number of nodes n. For the first time, we propose a precise normalization for a subset of instance features proposed in the literature which proved to have discriminating power regarding solver performances: features based on the node-depth in a minimum spanning tree and features derived from weakly/strongly connected components in a k-nearest neighbor transformation of the problem instance. Rigorous theoretical results give precise lower and upper bounds for the respective feature values which serve for unity-based normalization of each feature.

An empirical comparison of normalized with corresponding unnormalized features on a wide collection of commonly accepted TSP benchmark sets from the literature shows that normalization can successfully eliminate the effects of the instance size and facilitates the analysis of solver behavior based on instance characteristics. In addition, a proof-of-concept AS study highlights the potential

of feature normalization. Here, algorithm selectors with normalized input, based on support vector machines and random forests, outperform the respective models based on unnormalized feature input with respect to the penalized average runtime (PAR10) and managed to close the gap to the VBS substantially. This is especially noteworthy as we so far operated on a highly reduced feature set compared to the huge amount of TSP features available in general.

Therefore, a straight-forward extension of this work will tackle the derivation of precise normalizations for the complete set of TSP features which will form the basis of a comprehensive AS study on a large variety of instance sets. Deep learning based approaches will also be included together with alternative performance indicators such as PQR10 and multi-objective variants. In addition, we will work on a more sound understanding of solver behavior and respective differences related to instance (set) characteristics in order to improve AS model performance on TSP in general. There is a strong need for complementing existing feature sets with even more informative features based on insights gained so far.

ACKNOWLEDGMENTS

The authors acknowledge support by the European Research Center for Information Systems (ERCIS).

REFERENCES

[1] David L. Applegate, Robert E. Bixby, Vasek Chvátal, and William J. Cook. 2006. *The Traveling Salesman Problem.* Princeton University Press. https://doi.org/10.1515/9781400841103

[2] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Thomas Marius Lindauer, Yuri Malitsky, Alexandre Fréchette, Holger H. Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. 2016. ASlib: A Benchmark Library for Algorithm Selection. *Artificial Intelligence (AIJ)* 237 (2016), 41 – 58. https://doi.org/10.1016/j.artint.2016.04.003

[3] Jakob Bossek. 2017. *salesperson: Computation of Instance Features and R Interface to the State-of-the-Art Exact and Inexact Solvers for the Traveling Salesperson Problem.* https://github.com/jakobbossek/salesperson R package version 1.0.0.

[4] Jakob Bossek, Katrin Casel, Pascal Kerschke, and Frank Neumann. 2020. The Node Weight Dependent Traveling Salesperson Problem: Approximation Algorithms and Randomized Search Heuristics. In *Proceedings of the 22nd Annual Conference on Genetic and Evolutionary Computation* (Cancún, Mexico) *(GECCO '20)*. ACM, 1286 – 1294. https://doi.org/10.1145/3377930.3390243

[5] Jakob Bossek, Pascal Kerschke, Aneta Neumann, Markus Wagner, Frank Neumann, and Heike Trautmann. 2019. Evolving Diverse TSP Instances by Means of Novel and Creative Mutation Operators. In *Proceedings of the 15th ACM/SIGEVO Conference on Foundations of Genetic Algorithms* (Potsdam, Germany) *(FOGA '19)*. Association for Computing Machinery, 58–71. https://doi.org/10.1145/3299904.3340307

[6] Jakob Bossek and Heike Trautmann. 2016. Evolving Instances for Maximizing Performance Differences of State-of-the-Art Inexact TSP Solvers. In *Proceedings of the 10th International Conference on Learning and Intelligent Optimization (LION)* (Ischia, Italy). Springer, 48 – 59. https://doi.org/10.1007/978-3-319-50349-3_4

[7] Jakob Bossek and Heike Trautmann. 2016. Understanding Characteristics of Evolved Instances for State-of-the-Art Inexact TSP Solvers with Maximum Performance Difference. In *Advances in Artificial Intelligence (AI*IA)* (Genova, Italy). Springer, 3 – 12. https://doi.org/10.1007/978-3-319-49130-1_1

[8] Jérémie Dubois-Lacoste, Holger H. Hoos, and Thomas Stützle. 2015. On the Empirical Scaling Behaviour of State-of-the-Art Local Search Algorithms for the Euclidean TSP. In *Proceedings of the 17th Annual Conference on Genetic and Evolutionary Computation* (Madrid, Spain) *(GECCO '15)*. Association for Computing Machinery, 377–384. https://doi.org/10.1145/2739480.2754747

[9] Keld Helsgaun. 2009. General k-Opt Submoves for the Lin–Kernighan TSP Heuristic. *Mathematical Programming Computation* 1 (10 2009), 119–163. https://doi.org/10.1007/s12532-009-0004-6

[10] G. E. Hinton and R. R. Salakhutdinov. 2006. Reducing the Dimensionality of Data with Neural Networks. *Science* 313, 5786 (2006), 504–507. https://doi.org/10.1126/science.1127647

[11] Frank Hutter, Lin Xu, Holger Hoos, and Kevin Leyton-Brown. 2014. Algorithm Runtime Prediction: The State of the Art. *Artificial Intelligence* 206 (11 2014), 79 – 111. https://doi.org/10.1016/j.artint.2013.10.003

[12] Pascal Kerschke, Jakob Bossek, and Heike Trautmann. 2018. Parameterization of State-of-the-Art Performance Indicators: A Robustness Study Based on Inexact TSP Solvers. In *Proceedings of the 20th Annual Conference on Genetic and Evolutionary Computation Companion* (Kyoto, Japan) *(GECCO '18)*. ACM, 1737 – 1744. https://doi.org/10.1145/3205651.3208233

[13] Pascal Kerschke, Holger H. Hoos, Frank Neumann, and Heike Trautmann. 2019. Automated Algorithm Selection: Survey and Perspectives. *Evolutionary Computation (ECJ)* 27, 1 (2019), 3 – 45. https://doi.org/10.1162/evco_a_00242

[14] Pascal Kerschke, Lars Kotthoff, Jakob Bossek, Holger Hoos, and Heike Trautmann. 2018. Leveraging TSP Solver Complementarity through Machine Learning. *Evolutionary Computation* 26, 4 (2018), 597 – 620. https://doi.org/10.1162/evco_a_00215

[15] Lars Kotthoff. 2014. Algorithm Selection for Combinatorial Search Problems: A Survey. *AI Magazine* 35, 3 (2014), 48 – 60. https://doi.org/10.1007/978-3-319-50137-6_7

[16] Lars Kotthoff, Pascal Kerschke, Holger Hoos, and Heike Trautmann. 2015. Improving the State of the Art in Inexact TSP Solving Using Per-Instance Algorithm Selection. In *Learning and Intelligent Optimization*. Springer International Publishing, 202–217. https://doi.org/10.1007/978-3-319-19084-6_18

[17] Shen Lin and Brian W. Kernighan. 1973. An Effective Heuristic Algorithm for the Traveling-Salesman Problem. *Operations Research* 21, 2 (04 1973), 498–516. https://doi.org/10.1287/opre.21.2.498

[18] Olaf Mersmann, Bernd Bischl, Heike Trautmann, Markus Wagner, Jakob Bossek, and Frank Neumann. 2013. A novel feature-based approach to characterize algorithm performance for the traveling salesperson problem. *Annals of Mathematics and Artificial Intelligence* 69, 2 (2013), 151–182. https://doi.org/10.1007/s10472-013-9341-2

[19] Yuichi Nagata and Shigenobu Kobayashi. 1997. Edge Assembly Crossover: A High-power Genetic Algorithm for the Traveling Salesman Problem. In *Proceedings of the Seventh International Conference on Genetic Algorithms (ICGA 1997)*. Morgan-Kaufmann, San Francisco, CA, 450–457.

[20] Yuichi Nagata and Shigenobu Kobayashi. 2013. A Powerful Genetic Algorithm Using Edge Assembly Crossover for the Traveling Salesman Problem. *INFORMS Journal on Computing* 25 (05 2013), 346–363. https://doi.org/10.1287/ijoc.1120.0506

[21] Samadhi Nallaperuma, Markus Wagner, Frank Neumann, Bernd Bischl, Olaf Mersmann, and Heike Trautmann. 2013. A Feature-Based Comparison of Local Search and the Christofides Algorithm for the Travelling Salesperson Problem. In *Proceedings of the 12th ACM/SIGEVO Conference on Foundations of Genetic Algorithms* (Adelaide, Australia) *(FOGA '13)*. ACM, 147 – 160. https://doi.org/10.1145/2460239.2460253

[22] Josef Pihera and Nysret Musliu. 2014. Application of Machine Learning to Algorithm Selection for TSP. In *Proceedings of the 2014 IEEE 26th International Conference on Tools with Artificial Intelligence (ICTAI '14)*. IEEE Computer Society, USA, 47 – 54. https://doi.org/10.1109/ICTAI.2014.18

[23] R Core Team. 2020. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. https://www.R-project.org/

[24] Gerhard Reinelt. 1991. TSPLIB–A Traveling Salesman Problem Library. *ORSA Journal on Computing* 3, 4 (1991), 376–384. https://doi.org/10.1287/ijoc.3.4.376

[25] John R. Rice. 1976. The Algorithm Selection Problem. In *Department of Computer Science Technical Report*. Advances in Computers, Vol. 15. Elsevier, 65 – 118. https://doi.org/10.1016/S0065-2458(08)60520-3

[26] Gabriel Robins and Jeffrey S. Salowe. 1994. On the Maximum Degree of Minimum Spanning Trees. In *Proceedings of the Tenth Annual Symposium on Computational Geometry* (Stony Brook, New York, USA) *(SCG '94)*. Association for Computing Machinery, New York, NY, USA, 250–258. https://doi.org/10.1145/177424.177978

[27] Alireza Sarveniazi. 2014. An Actual Survey of Dimensionality Reduction. *American Journal of Computational Mathematics* 4 (2014), 55–72. https://doi.org/10.4236/ajcm.2014.42006

[28] Moritz Seiler, Janina Pohl, Jakob Bossek, Pascal Kerschke, and Heike Trautmann. 2020. Deep Learning as a Competitive Feature-Free Approach for Automated Algorithm Selection on the Traveling Salesperson Problem. In *Parallel Problem Solving from Nature – PPSN XVI*. Springer International Publishing, Cham, 48–64. https://doi.org/10.1007/978-3-030-58112-1_4

[29] Kate Smith-Miles, Jano Ilja van Hemert, and Xin Yu Lim. 2010. Understanding TSP Difficulty by Learning from Evolved Instances. In *Proceedings of the 4th International Conference on Learning and Intelligent Optimization (LION)* (Venice, Italy), Vol. 6073. Springer, 266 –280. https://doi.org/10.1007/978-3-642-13800-3_29

[30] Jarkko Venna and Samuel Kaski. 2005. Local multidimensional scaling with controlled tradeoff between trustworthiness and continuity. In *In Proceedings of 5th Workshop on Self-Organizing Maps* (Paris, France). 695 – 702.